

Just-Tech-IT

Chatbots

+

NLP

+

Artificial Intelligence

+

JavaScript

+

Conversational AI

+

Getting started with NLP.js



Jesús Seijas

Follow

7 min read · Sep 22, 2020



187



Ever wanted to build a chatbot and encountered some blockers along the way relating to data privacy or supported languages? Do you wish to reduce chatbot response time or run them without an active data connection?

If that's the case or if you're just curious and want to learn more, give [NLP.js](#) a try.

Natural Language Processing & NLP.js

Natural Language Processing or NLP is a field combining linguistics and computing, as well as artificial intelligence. Correctly understanding natural language is critical for virtual assistants, chatbots, voice assistants, and a wide range of applications based on a voice or text interface with a machine. These applications typically include a Natural Language Processor whose purpose is to extract the interactions and intention, as well as related

information and metadata, from a piece of plain natural language and translate them into something a machine can process.

NLP.js is an on-premise open source set of more than 70 libraries, used to tackle and solve the main three areas of NLPs: natural language understanding, language generation, and named entity recognition. The key differentiating feature that NLP.js provides is an enhanced user experience via an improved response time, additional language support and, according to some benchmarks, improved accuracy while leveraging increased data privacy & security controls and choices.

Why have an NLP library?

It isn't easy to understand how existing NLPs process every sentence and why specific behavior results as an output. This black box effect, due to the lack of visibility on why the chatbot has answered in a specific way without being able to dig into the source of the problem, causes frustration to chatbot managers. Having the NLP as an open-source library provides more visibility and understanding of the low-level natural language processing. It would enable technical people to better comprehend the processing of the conversation for managing language-specific strategies to achieve the expected accuracy level. Even if having a specific strategy per country isn't a mandatory approach, it's highly recommended when you target high-performance chatbots in languages other than the most-commonly used.

The main features of NLP.js

1. Language support

NLP.js supports up to 104 different languages with the use of BERT embeddings. Without BERT, it natively supports 41 languages.

2. Stemmers

NLP.js implements stemmers to both improve accuracy and require fewer training utterances to achieve the same result. It drastically reduces the manpower and computing power needed to train the NLP.

Stemmers are algorithms used to calculate the stem (root) of words. For example, words such as ‘developed’, ‘developer’, ‘developing’, ‘development’, and ‘developers’, are all classified as having the same stem — ‘develop’. This is important because when preparing sentences to be trained or classified by an NLP, we usually tend to split those sentences into features. Some NLPs use a tokenizer to divide them into words, but the problem with this approach is that you may need to train the NLP with more sentences to include the different inflections of the language.

Consider the example where you train the NLP with the sentence ‘who’s your developer?’ with the word ‘developer’ as the intent, and then, someone asks the question: ‘who developed you?’. Without a stemmer, the words ‘developer’ and ‘developed’ wouldn’t be recognized as being similar, as they aren’t identified with the same token. This issue is even more pronounced in highly inflected languages like Spanish or Indonesian, where the same word can be inflected to indicate gender or, in the case of verbs, tense, mood, and person for example.

3. Open questions

As a result of the integration with BERT, you can have open questions over texts using NLP.js. This means that instead of training the NLP with sentences and intents, you only have to provide a text to BERT and you could then ask any question over the text. The NLP.js BERT integration makes it

possible to have an unsupervised classification where you don't have to provide the intents.

Below, you can see an example where the text provided to the chatbot is information about Harry Potter, with some open questions subsequently asked over text:



4. Entity extraction

NLP.js enables entity extraction at several levels. It includes an optimized named entity extraction that can search and compare millions of possibilities in milliseconds.

Also, it has golden entity extraction to identify numbers, emails, phone numbers, measures, URLs, currency, etc. When we're talking about identifying a number, it can be quite simple when the figure is written in numerical digits such as '541', but it isn't so obvious to understand that 'five hundred and forty-one' corresponds to the same number. Currencies and measurements written in characters is possible for up to 44 languages in NLP.js.

NLP.js helps to optimize the user experience

Data privacy, security, and response time are key pillars for improving user experience and the overall conversational system.

Data privacy

Most of the NLP market leaders are cloud-based solutions, meaning that all the data is being processed in the cloud and, in some cases, managed outside of the target customer platform. In principle, cloud data processing isn't a big issue when aiming to meet the data privacy needs and requirements of most countries. However, it can still be a showstopper in certain regions, such as Germany, Singapore, or Turkey...

Security

The idea of making the NLP a library would allow the overall solution to be deployable fully on-premise if required. Furthermore, NLP.js could be

executed directly on a smartphone without needing a data connection. With the current trends of globalization and making everything more and more connected, it's important to keep an open door to fully on-premise solutions to maintain control over data..

Response time

By removing the need for cloud connectivity, a significant improvement in terms of latency and performance will be observed, even though, any API call will always have some inherent latency. This latency can be further avoided by including NLP.js as an embedded library. In terms of benchmarking, this faster performance would highlight a significant difference against other market solutions.

Running NLP.js locally (example)

First, you'll need Node.js installed on your computer. If you haven't, you can get it [here](#).

Get Jesús Seijas's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

Then, create a folder for your project, init a new node project and install these NLP.js dependencies: `basic`, `express-api-server` and `directline-connector`. `basic` installs the packages needed to run NLP.js, `express-api-server` provides an API server using `express` and the frontend for the chatbot, and `directline-connector` provides an API for the chatbot like the Microsoft Directline one.

```
mkdir chatbot
cd chatbot
npm init
npm i @nlpjs/basic @nlpjs/express-api-server @nlpjs/directline-connector
```

Now you'll need a Corpus, that's the knowledge data for your chatbot, organized into intents, and for each intent the sentences to train as well as the answers. You can access an example of corpus in English [here](#) or the [raw file](#). Download it and put it inside the folder where you've your project.

```
curl -O https://raw.githubusercontent.com/axa-group/nlp.js/master/examples/03-qna-pipelines/corpus.json
```

Create a file called *conf.json*, this is the configuration file telling NLP.js what plugins it must include and the configuration for each plugin. Put the following information in the *conf.json* file to run this example:

```
{
  "settings": {
    "nlp": {
      "corpora": ["/corpus.json"]
    },
    "api-server": {
      "port": 3000,
      "serveBot": true
    }
  },
  "use": ["Basic", "ExpressApiServer", "DirectlineConnector"]
}
```

The use part is the name of the plugins to include and the settings part is the configuration of each plugin. In this case we're telling the NLP to load

the corpora, the `corpus.json` file we downloaded before. We're also telling the API server to start on the port 3000 and we set `serveBot` to true as we want the frontend of the bot to be automatically served.

Now that we've the configuration, let's create an `index.js` file with the code to

Open in app ↗

Sign up

Sign in

Medium

Search

Write



```
const { dockStart } = require("@nlpjs/basic");

(async () => {
  const dock = await dockStart();
  const nlp = dock.get('nlp');
  await nlp.train();
})();
```

And that's everything we need. We can now start the application:

With `const dock = await dockStart()` we're telling NLP.js to initialize, load the `conf.json` file, load the associated plugins defined and start them with the defined configuration. It returns a dock instance that holds a container with all the plugins loaded. Then `const nlp = dock.get('nlp')` is where we retrieve the NLP plugin from the dock container. This instance of NLP already contains the corpus that we defined in the configuration, but isn't trained yet, so we've to train it with `await nlp.train()`.

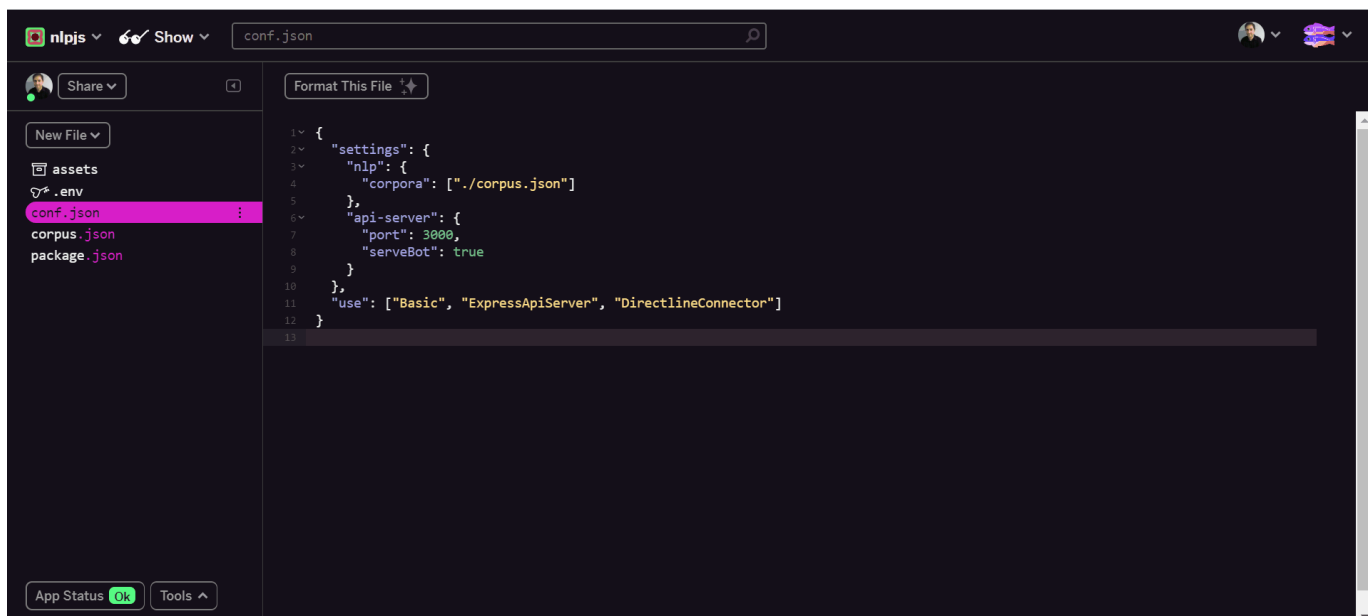
And that's everything we need. We can now start the application:

```
node .
```

And navigate to <http://localhost:3000> to see the webchat and talk with the chatbot.

Online demo

If you prefer to play with an online demo, you can ‘Remix’ the code on [Glitch](#), meaning you’ll be able to run the demo, as well as make your modifications to the code and play with it.



```
1 {
2   "settings": {
3     "nlp": {
4       "corpora": ["/corpus.json"]
5     },
6     "api-server": {
7       "port": 3000,
8       "serveBot": true
9     }
10  },
11  "use": ["Basic", "ExpressApiServer", "DirectlineConnector"]
12 }
13
```

For more information, you can access the [full tutorial](#) and some additional [codes snippets](#).

The value of open source

According to Tom Preston-Werner — cofounder of GitHub: “Smart people like to hang out with other smart people. Smart developers like to hang out with smart code. When you open source useful code, you attract talent”.

In our ambition to become a tech-led company, sharing relevant open-source projects and libraries is an excellent method to showcase our

technology to the world, extend our collaboration beyond our company walls, and to expand our ways of connecting with additional talent.

NLP.js is an excellent candidate for AXA's open-source program. It doesn't contain anything specific from the AXA core business, it's generic enough, easy to be reused, and we believe it provides a perfect opportunity to engage and contribute back to the open source community.

Among other uses and publications, it has already been used in the University of Goettingen and presented at the Colombia 4.0 AI conference in 2019.

If you want to learn more about AXA's open source program and technology, please contact: opensource@axa.com

Chatbots

+

NLP

+

Artificial Intelligence

+

JavaScript

+

Conversational AI

+



Written by Jesús Seijas

174 followers · 31 following

New Tech Team Lead at AXA Group Operations. Chatbot and voice agent expert, AI advocate.

[Follow](#)

Published in Just-Tech-IT

187 followers · Last published Jul 17, 2026

Sharing knowledge and best practices for valuable tech workers. This blog is Powered by @AXAJobs_fr

[Follow](#)

